Fast Parallel Image Rotation Algorithm

Dhrubajyoti Mandal
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22053859@kiit.ac.in

Dr. Sujoy Dutta
Asst. Professor & Asst. CoE
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
sdattafcs@kiit.ac.in

Sourav Kumar Parida
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22051032@kiit.ac.in

Chiranjeev Bhaya
Lead Engineer (Camera Systems)
Samsung R&D Institute
Bangalore, India
c.bhaya@samsung.com

Subhadeep Bhadra
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
2205336@kiit.ac.in

Chandra Mohan V
Architect (Camera Systems)
Samsung R&D Institute
Bangalore, India
chandra.mv@samsung.com

***Abstract*—This paper introduces an innovative single-pass algorithm for image rotation that greatly improves computational efficiency by treating entire rows or columns of pixels as cohesive vectors instead of mapping each pixel individually. The method dynamically selects between row-major and column-major operations based on the angle of rotation, ensuring optimized performance across a broad range of rotation scenarios. By determining the initial coordinates of each pixel vector and employing simple floating-point arithmetic for subsequent pixel positioning, the algorithm reduces the complexity of the rotation process while minimizing interpolation errors. This approach proves especially advantageous for high-resolution images and real-time applications, such as video processing and computer graphics, where both speed and image quality are critical. The proposed method offers a scalable, efficient, and precise solution for image rotation, marking a significant contribution to the field of digital image processing.**

***Index terms:* Image rotation, line rotation, image transformation, double-line rotation (DLR), and parallel image rotation.**

I. Introduction

2-D Image Rotation has a number of applications in digital photography and image processing like image matching, alignment, [1] etc. Precise image rotation is critical for specific image processing tasks like feature extraction and matching. Although numerous advanced image rotation algorithms exist, they often face challenges related to image quality and processing speed [2] [3]. Parallel algorithms for image processing aim to execute faster than sequential algorithms, however designing and scheduling a fast parallel image rotation is a challenge [4]. The most straightforward, and perhaps the most well known approach to implement image rotation is by using the rotation matrix [5], as described here

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x - cx \\ y - cy \end{bmatrix} * \begin{bmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{bmatrix}$$

Here, θ represents the rotation angle, while x' and y' denote the transformed coordinates of the rotated image. The point (cx, cy) corresponds to the center of the original image, which serves as the axis of rotation. If the center of rotation for the rotated image is given by (cx', cy'), the resulting coordinates are:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x' + cx' \\ y' + cy' \end{bmatrix}$$

II. Literature Review

Affine transformation is a widely used geometric transformation in image processing, which allows for rotation, translation, scaling, and shearing while preserving parallelism and collinearity of lines. It is defined by a 3x3 matrix that is applied to each point in the image. The standard equation for performing an affine transformation on a point (x, y)in an image is given by:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} cos(\theta) & -sin(\theta) & t_x \\ sin(\theta) & cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In this matrix, θ denotes the rotation angle, while $t_x and t_y$ indicate the translation along the x-axis and y-axis, respectively. The first two rows of the matrix determine the rotation

and scaling, while the third row is used for homogeneous coordinates, allowing translation.

In the context of image rotation, the matrix $\mathbf{A}$ is typically determined using trigonometric functions based on the rotation angle θ, while the translation components t_x and t_y are calculated to center the rotated image within its new dimensions. The `warpAffine` function in OpenCV is commonly used for performing affine transformations efficiently, providing flexibility in choosing different interpolation techniques such as nearest-neighbor, bilinear, and bicubic. These interpolation methods play a key role in reducing distortions and maintaining image quality during the transformation process [6].

However, affine transformations are not without limitations, especially when dealing with large rotation angles or combined transformations, such as scaling and shearing, which can lead to artifacts like aliasing and loss of detail.

To mitigate these issues, more sophisticated methods have been developed. One such method is the Hermite polynomial expansion, which leverages the orthogonality of Hermite polynomials to achieve smooth and accurate interpolation. This technique is particularly effective for image processing tasks requiring high precision. The image function is expanded using two-dimensional Hermite polynomials, and rotation is performed by transforming the coordinates with a rotation matrix. The rotated image is then reconstructed using inverse Hermite expansion, ensuring smooth transitions and minimal distortion. Despite its accuracy, the Hermite expansion method is computationally intensive, requiring the evaluation and summation of multiple polynomial terms, which can result in slower processing times. Additionally, while this method excels at smooth interpolation, it may struggle to maintain fine details in images with high-frequency content [3].

Another advanced technique is the Double Line Rotation (DLR) method, which is recognized for its ability to maintain geometric and intensity integrity during transformation. The DLR method divides the image into eight distinct zones, each governed by specific rotation equations and transformations. This zonal technique guarantees that every pixel from the original image is mapped to two corresponding pixels in the rotated image, preserving a consistent distance ratio between points. This characteristic is vital for applications requiring feature extraction invariance, making the DLR method highly effective for tasks in pattern recognition and computer vision. However, the complexity of managing multiple zones and their corresponding transformations can lead to inefficiencies, especially with high-resolution images. Moreover, distortions may occur at the boundaries between zones, potentially degrading the visual quality of the rotated image [2].

III. Proposed Method

The proposed method is a single pass algorithm, which means that it performs the entire image rotation operation at once. The proposed algorithm predominantly affects pixels in the row major (width), or ones in the column major (height), depending on the angle of rotation, α. If $\alpha \in \left(\frac{\pi}{4}, \frac{3\pi}{4}\right] \bigcup \left(\frac{5\pi}{4}, \frac{7\pi}{4}\right]$, rotation is performed in the ***column major***, and it is performed in the ***row major*** for the rest of the angles. Accordingly, for rotating the source image, entire row or column vectors of pixels are rotated as a single unit, instead of the more common approach, where each individual pixel is separately mapped. For this reason, it is essential to precisely identify the starting coordinates for the new row or column vector. The coordinates of the rest of the pixels in the vector can then be determined by simple floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

Apart from this, depending on α, the pixel processing order changes - i.e., whether the last row or column vector of pixels get processed first, or last. Since it has been found out that these values are very sensitive to α, the specific details of which are discussed in the later parts of this paper.

A. Rotation Of An Image In The Column Major

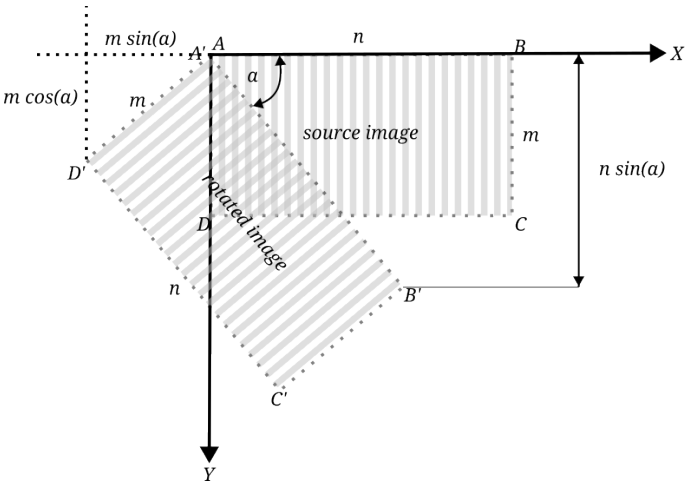

Fig. 1: Example demonstrating rotation of an image in the column major. Every column vector of pixels are rotated at once (Represented by the stripes in the image).

As seen in the example image in figure 1, for the given angle of rotation, column vectors were rotated (represented by the stripes). Each such vector to be rotated begins from the line $\overrightarrow{A'B'}$. As such, the initial coordinates of each of these vectors can be determined by the equation of the same line, which is expressed as:

$$f(i) = \left\lfloor \frac{i}{tan(\alpha)} \right\rfloor \forall \{i \in \mathbb{Z},\ i \in [0, n \cdot sin(\alpha))\} \qquad (1)$$

Thus, the initial coordinates of the column vectors would be of the type $(f(i), i)$. Next, to determine the coordinates of the rest of the pixels in the column vector, as mentioned previously, we perform floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

In the example in figure 1, we can see that each column vector is spread through a length of $m \cdot \sin(\alpha)$ in the x-axis and

through a length of $m \cdot \cos(\alpha)$ in the y-axis. It may also be observed that these values are the projections of any line perpendicular to line defined by the function in equation 1 on the coordinate axes.
We now define:

$$\delta x = |sin(\alpha)|$$

$$\delta y = |cos(\alpha)|$$

The coordinates of the next pixel in the column vector is then given by:

$$(f(i) + \delta x, i + \delta y)$$

The coordinates of the pixel following that is determined by adding δx and δy to the previous coordinates accordingly, and so on, for each of the m pixels in the given column vector. The end-result is that this operation covers all the $m \cdot \sin(\alpha)$ pixels in the x-axis and all the $m \cdot \cos(\alpha)$ pixels in the y-axis, therefore mapping all of the m pixels in the specific column vector.
This process continues for all of the n column vectors in the image, until the entire image is rotated. In practice, each column vector is actually mapped **twice**. The first mapping happens from the $i\,th$ column vector in I to the $i\,th$ column vector in R. The second mapping happens from the ith column vector in I to the $(i+1)\,th$ column vector in R. Each of these $(i+1)\,th$ column vectors get overwritten in each iteration with a new column vector in I; however, this gives a sufficiently accurate issusion of anti-aliasing without losing much quality.

The signs for δx and δy depend on whether, while mapping the column vectors in the rotated image, the values of the x-coordinate y-coordinate are decreasing or increasing. In the given example in figure 1, δx is negative, but δy is positive.
There is a third parameter, defined as:

$$\delta i = \frac{1}{\delta x}$$

δi determines the rate at which the column vector is processed from one to the next, and its sign depends on the angle of rotation, α. The calculations for δx, δy, and δi only happen once per column vector in the image, so the entire image transformation is quite fast, overall.

B. Rotation Of An Image In The Row Major

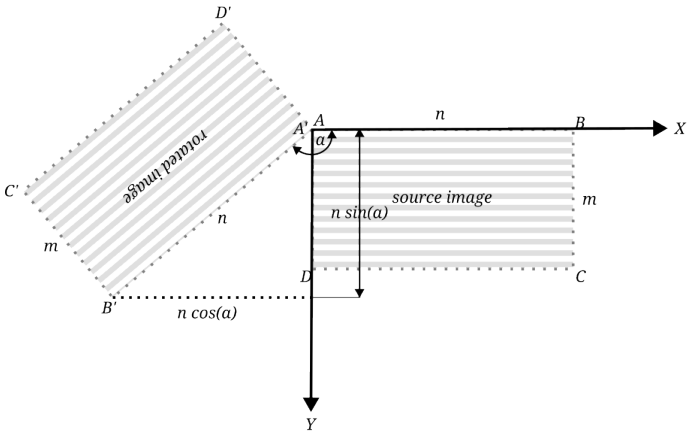

Fig. 2: Example demonstrating rotation of an image in the row major. Every row vector of pixels are rotated at once (Represented by the stripes in the image).

In the example image in figure 2, for the given angle of rotation, row vectors were rotated (represented by the stripes). All the calculations and logic is very similar to the one described for rotation in the column major [III-A]. Here, each vector to be rotated begins from the line $\overrightarrow{A'D'}$, whose equation is given by:

$$f(i) = \left\lfloor \frac{i}{tan(\alpha + \frac{\pi}{2})} \right\rfloor \forall \{i \in \mathbb{Z},\ i \in [0, m \cdot cos(\alpha))\} \quad (2)$$

Everything else stays the same, except the fact that now, each row vector is spread through a length of $n \cdot \cos(\alpha)$ in the x-axis and through a length of $n \cdot \sin(\alpha)$ in the y-axis, and accordingly, we have:

$$\delta x = |cos(\alpha)|$$

$$\delta y = |sin(\alpha)|$$

TABLE I: Notation

Symbol	Description
I	Original image
m, n	Height, width of I
R	Rotated image
m', n'	Height, width of R
α	Angle of rotation in radians
x'	Offset by which the x-coordinates of R need to be shifted
y'	Offset by which the y-coordinates of R need to be shifted
$f(...)$	Function defining the equation of the line from where the row or column vector starts [eq. 1] [eq. 2]
r_o	Upper limit of the function f
N	Number of elements (pixels) in the row or column vector f
`flip`	Boolean value that changes pixel processing order based on α
`row_major`	Boolean value that determines whether processing is to be done in the row or column major
`negate`	Boolean value, that if true, negates the indices used for indexing the pixels of the image

The algorithm described in [Algorithm 1] follows ***Python*** style indexing. For example, if `a = [[1, 2, 3], [5, 6]]`, then `a[0][-1]` represents the element `3`.

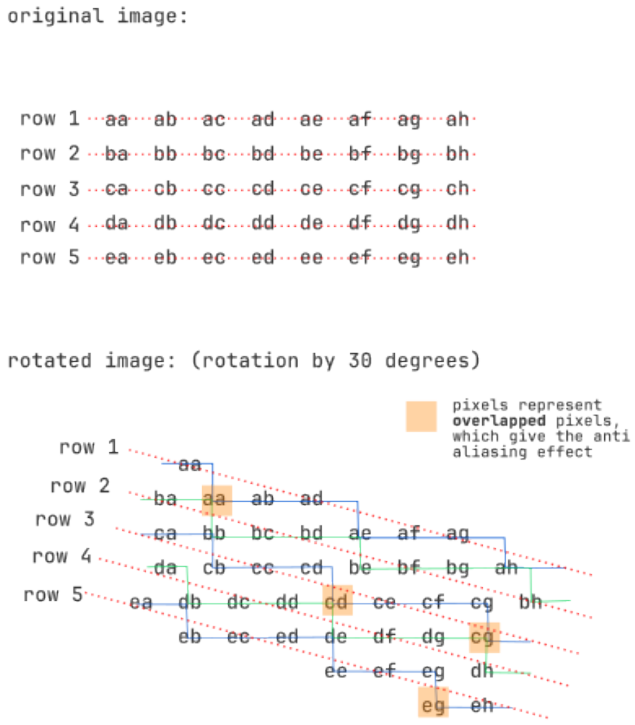

Fig. 3: Example demonstrating the proposed algorithm.

C. Implementation

Algorithm 1 Image Rotation Algorithm

procedure ROTATE(...) ▷ Performs image rotation on R
    `src_i` $= 0$ ▷ Initialize internal control variables
    `S` $= 1$
    $M = [\,]$
    if `negate` **then**
        `S = -1`
    end if
    if `flip` **then**
        $M = [m_i]\ for\ i = 0, 1, 2, \ldots, N-1, \mid m_i = N - i$
    else
        $M = [m_i]\ for\ i = 0, 1, 2, \ldots, N-1, \mid m_i = N$
    end if
    for $i = 0$ to r_o **do**
        `temp_x` $= f(i)$
        `temp_y` $= i$
        for px in M **do**
            $x = \lfloor$ `temp_x` $\rfloor$
            $y = \lfloor$ `temp_y` $\rfloor$
            `temp_i` $= \lfloor$ `src_i` $\rfloor$
            if `row_major` **then**
                $a =$ `temp_i`
                $b = px$
            else
                $a = px$
                $b =$ `temp_i`
            end if
            ▷ Map rotated pixels to R
            $R[$ `S` $*(y + y'),$ `S` $*(x + x')] = I[a, b]$
            $R[$ `S` $*(y + y') + 1,$ `S` $*(x + x')] = I[a, b]$
            `temp_x` $+= \delta x$
            `temp_y` $+= \delta y$
        end for
        `src_i` $+= \delta i$
    end for
end procedure

The algorithm described above [Algorithm 1] rotates image I to R.
R is Initialized as a blank image of width n' and height m' to fit R in the canvas, where:

$$m' = \lceil |m \cdot cos(\alpha) + n \cdot sin(\alpha)| \rceil$$

$$n' = \lceil |m \cdot sin(\alpha) + n \cdot cos(\alpha)| \rceil$$

The rotation function is then invoked, with each of the parameters updating depending on the value of α, as discussed above. The detailed values of each of the parameters are mentioned in [Table II].

It has been observed, that for ranges of α which are vertically opposite each other (For example, angles in the range $\left(\frac{\pi}{4}, \frac{\pi}{2}\right]$ are vertically opposite angles in the range $\left(\frac{5\pi}{4}, \frac{3\pi}{2}\right]$, and so on), the only parameters of rotation that change are δi, `negate`, and `flip`. As such, the calculations have been generalized for such angles and they have been represented in [Table II], with a ($'$) at the end of their names (shaded cells).

Algorithm 2 Driver Function

TABLE II: Rotation Parameters, based on α

Range of α	Rotation parameters											
	x'	y'	δx	δy	δi	r_o	**N**	`negate`	`flip`	$\delta i'$	`negate'`	`flip'`
$[0, \frac{\pi}{4}]$	$\lceil \lvert m \cdot \sin(\alpha) \rvert \rceil$	0	$\lvert \cos(\alpha) \rvert$	$\lvert \sin(\alpha) \rvert$	$1/\delta x$	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	n	`false`	`false`	$1/\delta x$	`true`	`false`
$(\frac{\pi}{4}, \frac{\pi}{2}]$	$\lceil \lvert m \cdot \sin(\alpha) \rvert \rceil$	0	$-\lvert \sin(\alpha) \rvert$	$\lvert \cos(\alpha) \rvert$	$-1/\delta x$	$\lceil \lvert n \cdot \sin(\alpha) \rvert \rceil$	m	`false`	`false`	$1/\delta x$	`false`	`true`
$(\frac{\pi}{2}, \frac{3\pi}{4}]$	m'	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	$-\lvert \sin(\alpha) \rvert$	$-\lvert \cos(\alpha) \rvert$	$-1/\delta x$	$\lceil \lvert n \cdot \sin(\alpha) \rvert \rceil$	m	`false`	`false`	$1/\delta x$	`false`	`true`
$(\frac{3\pi}{4}, \pi]$	$\lceil \lvert n \cdot \cos(\alpha) \rvert \rceil$	0	$-\lvert \cos(\alpha) \rvert$	$\lvert \sin(\alpha) \rvert$	$1/\delta x$	$\lceil \lvert m \cdot \cos(\alpha) \rvert \rceil$	n	`false`	`false`	$1/\delta x$	`true`	`false`

1: **procedure** MAIN(...) ▷ Invokes [Algorithm 1]
2: Determine the boolean value of `row_major`
3: Determine ***f***, based on the value of `row_major`
4: Initialize a blank image R with dimensions (m', n')
5: Determine the parameters of rotation from [Table II]
6: Generate R using [Algorithm 1]
7: **end procedure**

IV. Experimental Results and Analysis

We conduct an analysis of the accuracy of the reconstructed images through the use of different error metrics. These metrics assess the similarity between the original and processed images, offering valuable insights into the effectiveness of the image processing methods employed. The subsequent subsections provide a detailed explanation of each metric, including their mathematical formulations and interpretations. (All tests mave been measured against a sample image rotated by 26 degrees using GIMP.)

A. Image Data

The table below contains information about the different images used, for carrying out the experimental analysis.

TABLE III: Image Specifications

Image Name	Image Size (WxH)	Channels	Type	Format
licenseplate_motion.jpg	600x482	3	RGB	JPEG
squirrel_cls.jpg	535x426	3	RGB	JPEG
aero1.jpg	640x480	3	RGB	JPEG
apple.jpg	512x512	3	RGB	JPEG
baboon.jpg	512x512	3	RGB	JPEG
5am.jpg	3024x4032	3	RGB	JPEG
church_bottom.jpg	4032x2268	3	RGB	JPEG
church_wide.jpg	4032x2268	3	RGB	JPEG
corridor.jpg	2268x4032	3	RGB	JPEG
downhill.jpg	2268x4032	3	RGB	JPEG
landscape.jpg	1200x1200	3	RGB	JPEG
viewpoint.jpg	4032x2268	3	RGB	JPEG

B. Image quality analysis

1) Peak Signal-to-Noise Ratio (PSNR): PSNR is a commonly employed metric for assessing the quality of reconstructed or compressed images. It measures the ratio between the highest possible pixel value in the image and the mean squared error (MSE) between the original and processed images. PSNR is represented in decibels (dB), with higher values signifying improved image quality.

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{\text{MAX}_I}{\sqrt{\text{MSE}}} \right) \tag{3}$$

The Mean Squared Error (MSE) is determined using the formula:

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} \left[I(i,j) - K(i,j) \right]^2 \tag{4}$$

where MAX_I denotes the maximum possible pixel value (e.g., 65,535 for 16-bit images), while $I(i,j)$ and $K(i,j)$ represent the pixel values at the position (i,j) in the original and processed images, respectively.

TABLE IV: PSNR Error Analysis of Various Sample Images

Methods	Images				
	licenseplate_motion	squirrel_cls	aero1	apple	baboon
affine_transform	28.52	25.34	24.29	29.24	24.15
benchmark	31.49	25.22	25.94	31.65	24.60
bicubic	31.27	24.72	25.46	31.16	23.75
lanczos	31.23	24.68	25.43	30.99	23.58
linear	31.49	25.22	25.94	31.65	24.60
proposed_algorithm	28.71	24.49	24.92	30.06	22.85

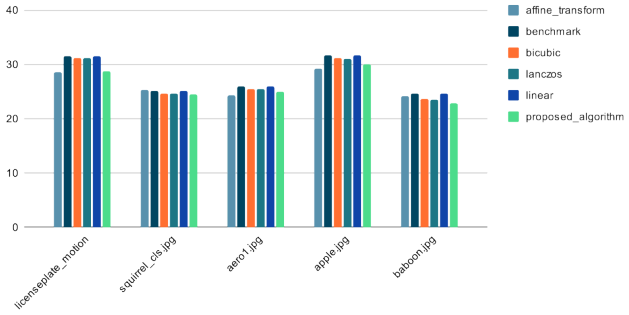

Fig. 4: PSNR analysis

2) Correlation Coefficient (CC): The Correlation Coefficient is a statistical measure that assesses the similarity between two images by analyzing the correlation of their corresponding pixels. Its value ranges from -1 to 1, with a value of 1 indicating perfect positive correlation, -1 signifying perfect negative correlation, and 0 representing no correlation whatsoever.

$$\mathrm{CC} = \frac{\sum_{i=1}^{M}\sum_{j=1}^{N}\left(I(i,j)-\overline{I}\right)\beta}{\sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N}\left(I(i,j)-\overline{I}\right)^2 \cdot \sum_{i=1}^{M}\sum_{j=1}^{N}\beta^2}} \tag{5}$$

where,

$$\beta = \left(K(i,j)-\overline{K}\right)$$

Here, $\overline{I}$ and $\overline{K}$ denote the average pixel values of the original and processed images, respectively.

TABLE V: CC Error Analysis of Various Sample Images

Methods	Images				
	licenseplate_motion	squirrel_cls	aero1	apple	baboon
affine_transform	0.9923	0.9835	0.9813	0.9911	0.9756
benchmark	0.9961	0.9830	0.9871	0.9949	0.9777
bicubic	0.9959	0.9810	0.9857	0.9943	0.9732
lanczos	0.9959	0.9808	0.9856	0.9940	0.9721
linear	0.9961	0.9830	0.9871	0.9949	0.9777
proposed_algorithm	0.9926	0.9800	0.9838	0.9926	0.9671

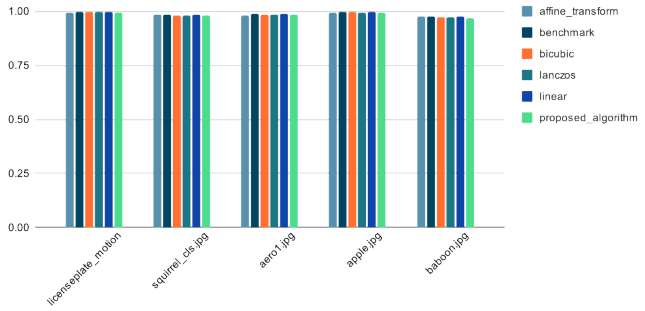

Fig. 5: CC analysis

C. Laplacian Mean Squared Error (LMSE)

Laplacian Mean Squared Error (LMSE) focuses on the high-frequency components of an image, particularly edges. By applying a Laplacian filter to both the original and processed images, LMSE calculates the error in edge details, which is particularly useful for detecting blurring or loss of fine details.

$$\mathrm{LMSE} = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}\left[\mathrm{Laplacian}(I(i,j)) - \mathrm{Laplacian}(K(i,j))\right]^2 \tag{6}$$

Lower LMSE values indicate less error in edge preservation, thus implying higher image quality.

TABLE VI: LMSE Error Analysis of Various Sample Images

Methods	Images				
	licenseplate_motion	squirrel_cls	aero1	apple	baboon
affine_transform	625.45	1473.12	1456.74	750.48	3571.89
benchmark	317.23	913.86	769.28	423.12	2460.18
bicubic	338.34	1119.33	901.44	489.30	3014.56
lanczos	346.34	1177.84	946.89	513.76	3168.34
linear	317.23	913.86	769.28	423.12	2460.18
proposed_algorithm	636.76	1645.89	1632.14	820.37	4638.00

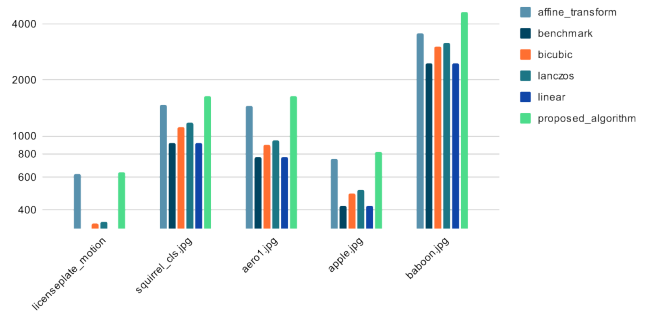

Fig. 6: LMSE analysis (scale is logarithmic)

1) Relative Error (Re): Relative Error (Re) measures the total difference between the original and processed images, normalized by the magnitude of the pixel values in the original

image. This metric is valuable for evaluating the overall accuracy of image reconstruction.

$$\text{Re} = \frac{\sum_{i=1}^{M}\sum_{j=1}^{N}|I(i,j) - K(i,j)|}{\sum_{i=1}^{M}\sum_{j=1}^{N}|I(i,j)|} \tag{7}$$

Lower values of Re indicate better similarity between the images.

TABLE VII: RE Error Analysis of Various Sample Images

Methods	Images				
	licenseplate_motion	squirrel_cls	aero1	apple	baboon
affine_transform	0.0204	0.0729	0.0723	0.0392	0.1022
benchmark	0.0254	0.0826	0.0672	0.0374	0.1056
bicubic	0.0259	0.0875	0.0718	0.0412	0.1169
lanczos	0.0262	0.0882	0.0721	0.0429	0.1197
linear	0.0254	0.0826	0.0672	0.0374	0.1056
proposed_algorithm	0.0266	0.0870	0.0732	0.0437	0.1253

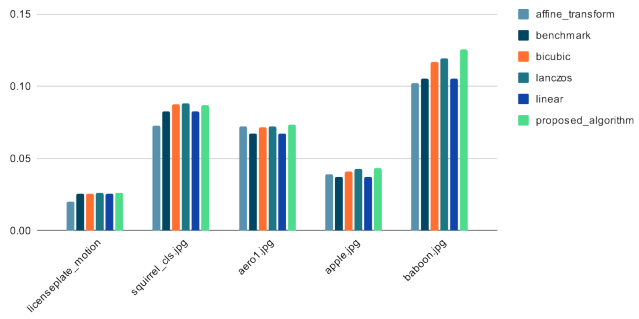

Fig. 7: RE analysis

2) Structural Similarity Index (SSIM): The Structural Similarity Index (SSIM) quantifies the perceptual differences between two images. Unlike metrics based on MSE, SSIM takes into account changes in structural information, luminance, and contrast, making it more reflective of human visual perception. SSIM values range from -1 to 1, where 1 indicates that the images are identical.

$$\text{SSIM}(I,K) = \frac{(2\mu_I\mu_K + C_1)(2\sigma_{IK} + C_2)}{(\mu_I^2 + \mu_K^2 + C_1)(\sigma_I^2 + \sigma_K^2 + C_2)} \tag{8}$$

In this equation, μ_I and μ_K represent the mean values, σ_I^2 and σ_K^2 denote the variances, and σ_{IK} is the covariance between the original and processed images. The constants C_1 and C_2 are incorporated to ensure stability in the formula when the denominator approaches zero, typically defined as:

$$C_1 = (k_1 L)^2, \quad C_2 = (k_2 L)^2 \tag{9}$$

where L signifies the dynamic range of the pixel values (e.g., 255 for 8-bit images), and k_1 and k_2 are small constants (e.g., $k_1 = 0.01$, $k_2 = 0.03$).

TABLE VIII: SSIM Error Analysis of Various Sample Images

Methods	Images				
	licenseplate_motion	squirrel_cls	aero1	apple	baboon
affine_transform	0.9722	0.8745	0.8204	0.8836	0.8345
benchmark	0.9689	0.8565	0.8309	0.8803	0.8127
bicubic	0.9674	0.8462	0.8175	0.8638	0.7980
lanczos	0.9670	0.8441	0.8159	0.8567	0.7922
linear	0.9689	0.8565	0.8309	0.8803	0.8127
proposed_algorithm	0.9633	0.8380	0.8062	0.8416	0.7619

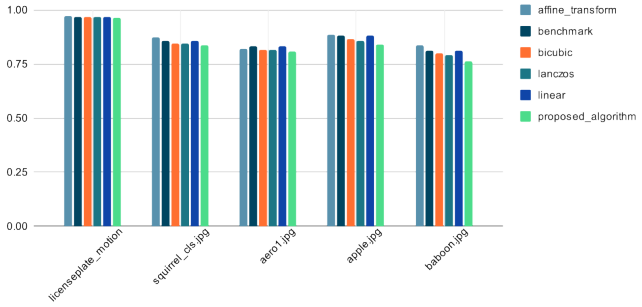

Fig. 8: SSIM analysis

D. Execution Time Analysis

Table IX shown below lists out the average execution time (in seconds) of each standard algorithm on various test images of sufficiently high resolutions. Each algorithm was run for a total of **36 times** - starting from rotaing the image from $0°$, to $360°$, in the intervals of $10°$. It has been noted that the proposed algorithm consistently surpasses the performance of existing techniques.

All the tests have been run on an `Intel i7-10750H (12) @ 5.000GHz` machine, with `16 GB` of RAM. All binaries were compiled for an `aarch64` device, and run on the same test machine, using **`qemu`**.

TABLE IX: Algorithm Performance Across Various Images

Algorithm	Images						
	5am.jpg	church_bottom.jpg	church_wide.jpg	corridor.jpg	downhill.jpg	landscape.jpg	viewpoint.jpg
affine_transform	2.806	2.206	2.231	2.225	2.168	0.543	2.32
benchmark	2.187	1.660	1.696	1.719	1.686	0.494	1.780
bicubic	2.107	1.605	1.645	1.669	1.632	0.478	1.724
lanczos	4.015	3.034	3.077	3.105	3.070	0.722	3.164
linear	2.278	1.720	1.748	1.775	1.783	0.502	1.831
proposed_algorithm	1.955	1.424	1.482	1.522	1.558	0.444	1.577

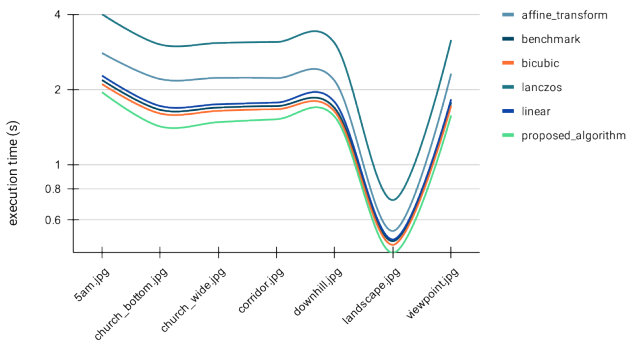

Fig. 9: Algorithm Performance Across Various Images (scale is logarithmic)

Table X presents a comparative analysis of various image rotation methods based on the number of multiplication, addition, and rounding-off operations. The proposed algorithm demonstrates a significantly lower computational burden, indicating its enhanced simplicity, which contributes to a substantial increase in processing speed.

TABLE X: Comparison of the Number of Multiplication, Addition, and Rounding-off Operations in Various Image Rotation Methods

Method	**Multiplication**	**Addition**	**Rounding-off**
Affine Transform	$4n^2$	$8n^2$	none
Benchmark	$4n^2$	$4n^2$	$2n^2$
Bicubic	$68n^2 + 4n$	$37n^2$	$2n^2$
Lanczos	$4n^2$	$4n^2$	$2n^2$
Linear	$4n^2 + 2n$	$4n^2 + 2n^2$	$2n^2$
DLR	$2n$	$4n^2 + 3n + 1$	$2n^2$
Proposed	n	$10n^2$	$2n^2 + n$

V. Conclusion

The proposed single-pass image rotation algorithm represents a significant advancement in the field of digital image processing by optimizing the way pixels are transformed during rotation. Traditional image rotation methods typically involve mapping each individual pixel to a new location based on the angle of rotation, which can be computationally expensive, especially for high-resolution images. In contrast, the proposed method processes entire rows or columns of pixels at once, greatly improving efficiency.

By treating pixel vectors as cohesive units, this approach reduces the complexity of the rotation operation, making it particularly useful for applications where large-scale image transformations are required in real time, such as video processing, computer graphics, and augmented reality.

The method's ability to determine the initial coordinates of the rotated vectors with precision and use simple floating-point arithmetic for subsequent pixel placement offers a streamlined and scalable solution. This makes the proposed algorithm not only computationally efficient but also versatile, allowing it to adapt to a wide range of rotation angles without significant performance trade-offs.

References

[1] B. Gaster, L. Howes, D. R. Kaeli, P. Mistry, and D. Schaa, *Heterogeneous computing with openCL: revised openCL 1*. Newnes, 2012.

[2] A. H. Ashtari, M. J. Nordin, and S. M. M. Kahaki, "Double line image rotation," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3370–3385, 2015.

[3] W. Park, G. Leibon, D. N. Rockmore, and G. S. Chirikjian, "Accurate image rotation using hermite expansions," *IEEE Transactions on Image Processing*, vol. 18, no. 9, pp. 1988–2003, 2009.

[4] H.-C. Kwon, H.-J. Cho, and H.-Y. Kwon, "A study on high speed image rotation algorithm using cuda," *The Journal of the Institute of Internet, Broadcasting and Communication*, vol. 16, no. 5, pp. 1–6, 2016.

[5] P. R. Evans, "Rotations and rotation matrices," *Acta Crystallographica Section D: Biological Crystallography*, vol. 57, no. 10, pp. 1355–1359, 2001.

[6] J. Wang and J. Watada, "Panoramic image mosaic based on surf algorithm using opencv," in *Proceedings of the IEEE*, pp. 1–7, 2021.